

Groovy DSL

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

- Who this book is for . . . 2
- A book that stands on its own . . . 2
- What you need . . . 3
- Conventions . . . 3

I: The Why . . . 4

- **1. Models as scripts** . . . 5
  - 1.1. A script is a model with a language around it . . . 5
  - 1.2. One DSL for every metamodel . . . 6
  - 1.3. Transcription and derivation . . . 6
  - 1.4. Code is executable . . . 6
- **2. Decisions** . . . 7
  - 2.1. Prior art . . . 7
  - 2.2. A trained eye . . . 7
  - 2.3. Reflective, not generated . . . 7
  - 2.4. References later . . . 8
  - 2.5. Files that say what they are . . . 8
  - 2.6. Read-only by default . . . 8
  - 2.7. Locations, not values . . . 8

II: The How . . . 9

- **3. Getting started** . . . 10
  - 3.1. Adding the module . . . 10
  - 3.2. File names . . . 10
  - 3.3. Loading a script . . . 11
  - 3.4. The generic DSL . . . 11
- **4. The language** . . . 13
  - 4.1. Entry points and dispatch . . . 13
  - 4.2. Three ways to set a feature . . . 14
  - 4.3. Negative numbers . . . 14
  - 4.4. Variables . . . 15
  - 4.5. What a script produces . . . 15
  - 4.6. Keywords . . . 15
- **5. References** . . . 17

5.1. Paths . . . 18
5.2. Names of classifiers . . . 19
5.3. Supersession . . . 19
5.4. Global objects . . . 20
6. Deriving models . . . **21**
6.1. Loading other resources . . . 21
6.2. Iterating a loaded resource . . . 22
6.3. Copying prototypes . . . 22
6.4. Containment is exclusive . . . 24
7. Saving . . . **25**
7.1. The truncation trap . . . 26
7.2. Patterns . . . 26
8. Errors and telemetry . . . **27**
8.1. Errors . . . 27
8.2. Telemetry . . . 27
9. A DSL for your metamodel . . . **30**
9.1. The metamodel . . . 30
9.2. A rule set . . . 31
9.3. The handler . . . 32
9.4. The capability factory . . . 33
9.5. Putting it together . . . 34
9.6. Resolvers . . . 35
III: Looking Ahead . . . **37**
10. What comes next . . . **38**
Appendix A: Reference . . . **39**
A.1. Types . . . 39
A.2. Precedence . . . 39
Appendix B: Building this book . . . **40**
Appendix C: Glossary . . . **41**

Colophon

Groovy DSL: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the Groovy module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:groovy:2026.11.0`, Java module `org.nasdanika.sdk.runtime.groovy`. Every listing in it is included from the module's sources, tests and test scripts, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Groovy DSL: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Apache Groovy and Apache are trademarks of the Apache Software Foundation. Eclipse, EMF and Xtext are trademarks of the Eclipse Foundation. Gradle is a trademark of Gradle, Inc. Kotlin is a trademark of JetBrains. Java is a trademark of Oracle and its affiliates. OpenTelemetry is a trademark of the Linux Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies discussed and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `groovy`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about writing models as Groovy scripts. It describes the Groovy module of the Nasdanika SDK Runtime, which loads a file such as `library.ecore.groovy` as an EMF resource: the script builds a tree of model objects, and the objects become the resource's contents. The language is derived from the metamodel by reflection, so every metamodel, Ecore itself included, has a DSL without anyone writing one.

The first part is the why: why models should be writable as scripts, and the decisions that shape the DSL. The second part is the how: the language, references, scripts that derive models and write themselves back, diagnostics, and binding the DSL to a metamodel of your own. The third says what comes next.

Who this book is for

Modelers who would rather write a model than click it together or read it as XMI: metamodels, architectures, rule sets, catalogs.

Developers who derive models from other models, diagrams, spreadsheets or systems, and want the derivation to be a file next to the model rather than a program somewhere else.

Tool builders who want a textual surface for their metamodel without a grammar, a parser or a code generator.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The runtime has a book of its own, *Nasdanika SDK Runtime: More Structure with Less Effort*, whose chapter on the Groovy DSL introduces what this book covers in depth. You do not need it to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.groovy`, Maven coordinates `org.nasdanika.sdk.runtime:groovy:2026.11.0`. It brings Apache Groovy 5.
- Some familiarity with EMF helps: a model is a tree of `EObject`s contained in a resource, typed by the `EClass`es of a metamodel.

Conventions

- **Every listing is included from the module's sources, tests or test scripts**, and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies between the module and its documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> This book was written from the module's readme, and replaces it: the readme was deleted when the book was complete.

Part I: The Why

Why models should be writable as scripts, and the decisions behind a DSL that every metamodel gets for free. No code is needed to read this part.

Chapter 1. Models as scripts

A model has to be written down somewhere, and every way of writing it down has a cost.

XMI is what EMF saves by default: complete, exact, and written for machines. A person can read a small XMI file, and nobody wants to write one.

YAML and JSON are friendlier, and the runtime reads models from both. They are data, though: no variables, no loops, no computation, and every reference spelled out as a path.

Diagrams are the friendliest of all for some models, an architecture or a process, and the runtime reads Draw.io diagrams as models. They are the wrong tool for a model of four hundred rules, or for one that is computed from another.

Editors generated from the metamodel, forms and trees, are good for changing one element and poor for seeing the whole, comparing two versions, or reviewing a change.

1.1. A script is a model with a language around it

A Groovy script that builds a model reads like a structured document: braces for containment, a call for each feature. It is also a program, so it can do what data cannot:

- **name things and refer to them**, with a variable instead of a path;
- **repeat and compute**: a loop for twenty similar elements, a function for a pattern used in ten places;
- **derive**: load a diagram, a spreadsheet or another model, and emit a model from it;
- **write itself back**: a script can decide what saving the model means, rewriting its own source or pushing the model to another system.

A script is text, so it is diffed, reviewed and merged like code, and versioned with the code that uses it.

1.2. One DSL for every metamodel

A DSL usually means work: a grammar, a parser, an editor, a code generator, for each language. This one is derived from the metamodel at load time. Every concrete class of the metamodel is a keyword that creates an instance of it; every feature is a call that sets it; containment is nesting. Bind the DSL to Ecore and you write metamodels; bind it to a rules metamodel and you write rule sets; bind it to an architecture metamodel and you write architectures. The metamodel changes, and the language follows on the next load, with nothing to regenerate.

1.3. Transcription and derivation

Most model files are transcriptions: someone knows the model and writes it down. Scripts can also be derivations: the model is computed from something else, and the script says how. A derivation stays true when its source changes, because it is run again rather than edited again. The two mix in one file: copy the structure from a diagram, and add by hand what only a person knows.

1.4. Code is executable

There is a price, and it must be said plainly. A `.groovy` model file is a program, and loading it runs it, with the permissions of the process that loads it. Load scripts you trust, from repositories you trust, as you would build files and test code. A model that arrives from an untrusted source belongs in a data format, YAML, JSON or XMI, which loads without running anything.

Chapter 2. Decisions

2.1. Prior art

Groovy builders for EMF. Jörn Dinkla described an `EMFBuilder` in 2007: nested closures over an EMF factory, braces for containment, Groovy variables for references. Groovy's own `BuilderSupport` and `FactoryBuilderSupport` are the general machinery, and this DSL is in that tradition.

Textual Ecore languages. Emfatic, Xcore and OCLinEcore are textual syntaxes for metamodels, and Xtext builds textual languages for any metamodel. They are generative: a grammar, a parser and an editor per language, built before the first model is written. This DSL is interpretive: no grammar to own and no generation step, and the host language's tooling, its editor support, its debugger, its libraries, comes with it.

Kotlin type-safe builders. Kotlin's builders, lambdas with receivers, check a DSL at compile time, and Gradle's Kotlin DSL shows them at scale. They are a candidate for a second surface (Chapter 10); Groovy came first because it is light, starts fast, and is dynamic enough to derive a language from a metamodel at run time.

2.2. A trained eye

Gradle's build files have taught a generation of Java developers to read Groovy builder syntax: a name, a block in braces, settings inside it. `ePackage { name 'library'; eClass { ⋯ } }` looks like a build file, and is read the same way. The DSL asks for no new reading skill, only for the metamodel's names in the places Gradle has its own.

2.3. Reflective, not generated

Everything the language accepts is read from the metamodel when a script is loaded:

- every concrete class whose decapitalized name is unambiguous is an **entry point**, `ePackage { }`;
- inside a builder, a call named after a **feature** of the current element sets it, `name 'library'`;
- a call named after a **class** creates a child and routes it into the one containment feature that

accepts it, `eClass { }`;

- a handful of **keywords**, `global`, `loadResource`, `copy`, `eObject` and others, are methods of the builder and win over both.

Nothing is generated, so nothing goes stale, and a dynamic metamodel, built or loaded at run time, works exactly as a generated one.

2.4. References later

A reference may point at an element written further down the file, or in another file. So references given as paths or closures are not resolved while the script runs: they are recorded with the line they were written on, and resolved after the whole script has built the model. A reference that cannot be resolved fails with that line.

2.5. Files that say what they are

A file name is a chain of qualifiers, read from the right: `library.ecore.groovy` is Groovy source, and the `ecore` qualifier binds it to Ecore. Each qualifier is served by a capability, found through the runtime's capability framework: the `groovy` qualifier by a handler that compiles the script, the `ecore` qualifier by a handler that evaluates it against the Ecore metamodel. A module that adds a metamodel adds a qualifier, `orders.rules.groovy`, by contributing a capability, and no other module changes (Chapter 9).

2.6. Read-only by default

A script is the source of truth, and there is no general way to turn an edited model back into the script that built it. So saving a script resource fails, unless the script registers what saving means for it (Chapter 7).

2.7. Locations, not values

Loading is traced with OpenTelemetry, a span per element and per feature, with the script line. The spans carry where something was written and what type it is, never the values: a model may hold data that a trace must not (Chapter 8).

Part II: The How

Adding the module, the language, references, scripts that derive models from other resources and write themselves back, errors and telemetry, and a DSL bound to a metamodel of your own.

Chapter 3. Getting started

3.1. Adding the module

The runtime has a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and their dependencies. Import it in `dependencyManagement` with the scope `import` and the type `pom`, and depend on `org.nasdanika.sdk.runtime:groovy` without a version. The module brings Apache Groovy and its JSR-223 script engine.

Listing 1. The module descriptor (`module-info.java`)

```
module org.nasdanika.sdk.runtime.groovy {

    exports org.nasdanika.sdk.runtime.groovy;
    exports org.nasdanika.sdk.runtime.groovy.dsl;
    opens org.nasdanika.sdk.runtime.groovy.dsl;

    requires transitive org.apache.groovy.jsr223;
    requires transitive org.apache.groovy;
    requires transitive org.nasdanika.sdk.runtime.common;
    requires transitive java.scripting;

    provides CapabilityFactory with
        GroovyCapabilityFactory;

    provides ResourceSetContributor with
        GroovyResourceFactoryContributor;

}
```

The module provides two services: a resource set contributor that registers a resource factory for the `groovy` extension, and a capability factory with the contents handlers that compile and evaluate scripts. Any resource set the runtime configures from the module path loads `.groovy` files.

3.2. File names

File	Qualifiers, from the right	Loaded as
`model.groovy`	`groovy`	The generic DSL: no entry points, any registered class through `eObject('<nsURI>#<Class>')`

File	Qualifiers, from the right	Loaded as
`library.ecore.groovy`	`groovy`, `ecore`	Ecore: `ePackage`, `eClass`, `eAttribute` and every other concrete Ecore class are entry points
`orders.rules.groovy`	`groovy`, `rules`	A metamodel of your own, bound by a capability you contribute (Chapter 9)

3.3. Loading a script

A script is loaded like any other resource:

Listing 2. Loading a metamodel written in Groovy (`LanguageTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(); ①
Resource resource = resourceSet.getResource(uri("library.ecore.groovy"), true); ②
EPackage library = (EPackage) resource.getContents().get(0);
assertThat(library.getName()).isEqualTo("library");
```

① A resource set configured from the module path, with the `groovy` resource factory.

② Loading the file evaluates the script; the model it builds is the resource's contents.

Files load from anywhere a URI handler reaches: `file:` URIs, `classpath://` and `module://` resources, HTTP, or the runtime's volumes.

3.4. The generic DSL

A plain `.groovy` file has no metamodel bound to it, so it has no entry points. `eObject('<nsURI>#<Class>') { }` creates an instance of any class of any package registered with the resource set:

Listing 3. A plain `.groovy` file (`plain.groovy`)

```
eObject('http://www.eclipse.org/emf/2002/Ecore#EPackage') {
    name 'plain'

    eObject('http://www.eclipse.org/emf/2002/Ecore#EClass') {
        name 'Thing'
    }
}
```

Listing 4. Loaded like any other

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource resource = resourceSet.getResource(uri("plain.groovy"), true);
EPackage plain = (EPackage) resource.getContents().get(0);
assertThat(plain.getEClassifier("Thing")).isInstanceOf(EClass.class);
```

The bound DSLs are the comfortable way; the generic one is there for packages nobody has bound yet.

Chapter 4. The language

The examples of this chapter and the next are a small metamodel of a library, written in the Ecore DSL: a library holds books and authors, and books and authors refer to each other.

4.1. Entry points and dispatch

Listing 5. The beginning of the library metamodel (`library.ecore.groovy`)

```
ePackage {                                          ①
    name 'library'                                  ②
    nsURI 'urn:example:library'
    nsPrefix 'library'
    global 'urn:example:library'                    ③

    def namedElement = eClass {                     ④
        name 'NamedElement'
        'abstract'(true)                            ⑤
        eAttribute { name 'name'; eType 'EString' }
    }
}
```

① An entry point: every concrete Ecore class with an unambiguous decapitalized name is one, and creates a root of the resource.

② Feature dispatch: `name` is a feature of `EPackage`, so the call sets it.

③ A keyword: registers the package as a global object, reachable by that URI from any resource of the resource set (Chapter 5).

④ Type dispatch: `eClass` is a class, so the call creates an `EClass` and routes it into `eClassifiers`, the one containment feature of `EPackage` that accepts it. The call returns the element, so a variable can hold it.

⑤ `abstract` is a Java keyword, so it cannot be called by its bare name; Groovy calls a method named by a string.

Inside a builder, a call is tried in this order:

1. a **keyword** of the builder: `global`, `proxy`, `loadResource`, `copy`, `eObject`;
2. a **feature** of the current element: the call sets it;
3. a **class**: the call creates an element of it and routes it into the containment feature that accepts it.

A class that fits more than one containment feature, or a feature typed by an abstract class, needs the feature named and the type given: `eClassifiers('EClass') { }`.

4.2. Three ways to set a feature

Form	Example	Groovy sees
Command call	`lowerBound 1`	A method call: `lowerBound(1)`
Parentheses	`lowerBound(1)`	A method call
Assignment	`lowerBound = 1`	A property assignment

All three set the feature, with the value converted to the feature's type: a string names a classifier for `eType`, a number becomes an `int`, a string names an enum literal.

4.3. Negative numbers

One form fails silently, and only for negative numbers:

Listing 6. Three ways to say "unbounded" (`forms.ecore.groovy`)

```
ePackage {
    name 'forms'

    eClass {
        name 'Shelf'
        eAttribute { name 'assigned'; eType 'EString'; upperBound = -1 }  // Sets -1
        eAttribute { name 'called'; eType 'EString'; upperBound(-1) }     // Sets -1
        eAttribute { name 'subtracted'; eType 'EString'; upperBound -1 }  // Sets nothing
    }
}
```

Listing 7. Only two of them work

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource resource = resourceSet.getResource(uri("forms.ecore.groovy"), true);
EPackage forms = (EPackage) resource.getContents().get(0);
EClass shelf = (EClass) forms.getEClassifier("Shelf");
assertThat(shelf.getEStructuralFeature("assigned").getUpperBound()).isEqualTo(-1);
assertThat(shelf.getEStructuralFeature("called").getUpperBound()).isEqualTo(-1);
assertThat(shelf.getEStructuralFeature("subtracted").getUpperBound()).isEqualTo(1); ①
```

① `upperBound -1` is the expression `upperBound - 1`: Groovy reads the feature, subtracts one, and

discards the result. The feature keeps its default.

Whenever a value starts with a minus or a plus sign, use the assignment or the parentheses. EMF's "many", an upper bound of -1, is the case that comes up every day.

4.4. Variables

A call that creates an element returns it, so a variable can hold it, and the variable can be used wherever the element is needed: as a supertype, as the type of a reference, as the target of one. A variable is the most direct reference there is: no path and no name to resolve, and the compiler catches a misspelled one.

4.5. What a script produces

The resource's contents come from what the script produces:

1. If the script's **last expression** is an `EObject`, an array or a collection of them, those.
2. Otherwise, the **roots** created by entry points, in order.

A script with one root needs no return; a script with several returns a list, `[ ePackage { }, ePackage { } ]`, or creates them with entry points and ends with something that is not a model element.

4.6. Keywords

Keyword	Where	Does
`eObject('<name>') { }`	Anywhere	Creates an instance of a class named in simple, qualified or URI form (Chapter 5)
`global '<uri>'`	Builder	Registers the current element as a global object
`proxy '<uri>'`	Builder	Makes the current element an EMF proxy for the URI, resolved by EMF when it is used
`ref '<selector>'`	Top level	Resolves a selector now, and returns the element
`loadResource '<uri>'`	Anywhere	Loads another resource through the resource set (Chapter 6)

Keyword	Where	Does
`copy('<selector>') { }`	Anywhere	A deep copy of a prototype, configured by the closure (Chapter 6)
`onSave { }`	Top level	What saving the resource means (Chapter 7)
`dsl`	Top level	The DSL context: its `roots`, and `dsl.copy` for a copy placed by hand

Keywords win over metamodel names. A feature with a keyword's name is still settable with the assignment form, `copy = 'value'`, and a class with a keyword's name is still creatable with `eObject('Copy') { }`.

Chapter 5. References

A non-containment reference points at an element that exists elsewhere: a supertype, the type of a feature, an opposite. Its value can be given in three ways.

Listing 8. The rest of the library metamodel (`library.ecore.groovy`)

```
eClass {
    name 'Library'
    eSuperTypes namedElement                    ①

    eReference {
        name 'books'
        eType "/eClassifiers[name='Book']"      ②
        containment true
        upperBound = -1                         ③
    }

    eReference {
        name 'authors'
        eType { eContainer().eContainer().getEClassifier('Author') } ④
        containment true
        upperBound(-1)
    }
}

eClass {
    name 'Book'
    eAttribute { name 'title'; eType 'EString' }
    eAttribute { name 'year'; eType 'EInt' }

    eReference {
        name 'authors'
        eType "../../eClassifiers[name='Author']"  ⑤
        upperBound = -1
        eOpposite "../../eClassifiers[name='Author']/eStructuralFeatures[name='books']"
    }
}

eClass {
    name 'Author'
    eSuperTypes namedElement

    eReference {
        name 'books'
        eType "../../eClassifiers[name='Book']"
        upperBound = -1
        eOpposite "../../eClassifiers[name='Book']/eStructuralFeatures[name='authors']"
    }
}
```

① An **element**, held in a variable: used as it is.

② A **path**, anchored at the root of the resource by the leading slash: the classifier named `Book`, written further down the file.

③ The assignment form, because the value is negative.

④ A **closure**, run after the script with the reference as its delegate and its argument: `eContainer()` is the `EClass`, its container the package. A closure may return an element or a path.

⑤ A **relative path**: `..` is the container of the current element, the `EClass`, and `../..` the package.

Listing 9. The model the script built

```
EClass namedElement = (EClass) library.getEClassifier("NamedElement");
EClass book = (EClass) library.getEClassifier("Book");
EClass author = (EClass) library.getEClassifier("Author");
assertThat(namedElement.isAbstract()).isTrue();

EClass libraryClass = (EClass) library.getEClassifier("Library");
EReference books = (EReference) libraryClass.getEStructuralFeature("books");
assertThat(books.getEType()).isSameAs(book); // Resolved after the script ran
assertThat(books.isMany()).isTrue();

EReference authors = (EReference) book.getEStructuralFeature("authors");
assertThat(authors.getEOpposite()).isSameAs(author.getEStructuralFeature("books"));
```

Paths and closures are resolved after the whole script has run, which is why `books` can refer to `Book` before `Book` is written.

5.1. Paths

A path is a sequence of EMF URI fragment segments separated by slashes:

- a **leading slash** anchors it at the root of the current element's tree; without one, it starts at the current element;
- `..` goes to the container, and `.` stays, which is useful as `./Name` to force a path where a name could also be a classifier (below);
- a segment names a feature and selects an element of it: by an attribute, `eClassifiers[name='Book']`, by position, `eClassifiers.2`, or by key. The `@` EMF puts in front of a feature name is optional;

- a path that contains `#` is a **URI**, resolved against the script's own URI and through the resource set: another resource, a global object, a proxy.

Predicate values are quoted. In a double-quoted Groovy string, single quotes need no escaping: `"/eClassifiers[name='Book']"`.

5.2. Names of classifiers

A value without a slash is tried first as the name of a classifier of the metamodel, which is how `eType 'EString'` finds Ecore's string type:

Form	Example	Resolves to
Simple	`'EString'`, `'eString'`	The classifier with that name, capitalized or not: in the current element's package first, then in all of them
Qualified	`'c4.Component'`	The trailing segment is the class; the leading ones are a package path, matched against the package's full path or any trailing part of it
URI	`'http://www.eclipse.org/emf/2002/Ecore#EClass'`	`<nsURI>#<Class>`: never ambiguous

A local element named like a metamodel classifier is reached with `./`: `eType "./Book"` is a path, even if the metamodel had a `Book`.

5.3. Supersession

When a simple name matches several classes and one of them is a subtype of another, the most specific wins. This is what makes refinements work: a governance model whose `ModelElement` extends the core model's `ModelElement` gets the governed one for `modelElement { }`, as an entry point, by type dispatch, and even inside an element of the core package. The superseded classes are recorded in the telemetry (Chapter 8). Two refinements of the same class are ambiguous, because neither is more specific, and a name that stays ambiguous fails with the qualified alternatives listed. Qualified and URI names select exactly what they name.

5.4. Global objects

`global '<uri>'` registers the current element with the resource set under a URI, so any resource of the set can refer to it by that URI, without knowing which file defines it:

Listing 10. A global object (`LanguageTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
resourceSet.getResource(uri("library.ecore.groovy"), true);
EObject library = resourceSet.getEObject(URI.createURI("urn:example:library"), false);
assertThat(library).isInstanceOf(EPackage.class); ①
```

① The library package, registered by `global 'urn:example:library'`, found by its URI alone.

Inside a script, a selector containing `#`, or `ref('<uri>')` at the top level, reaches a global object. `proxy '<uri>'` is the other direction: it turns the element being built into an EMF proxy for the URI, which EMF resolves when the element is used, so a model can refer to a global object that is loaded later.

Chapter 6. Deriving models

6.1. Loading other resources

`loadResource '<uri>'` loads another resource and returns it. With it a script stops being a transcription of a model and becomes a derivation of one:

Listing 11. A package of data transfer classes derived from the library (`derived.ecore.groovy`)

```
import org.eclipse.emf.ecore.EClass

def library = loadResource 'library.ecore.groovy'                ①
def classes = []
library.allContents.forEachRemaining {                           ②
    if (it instanceof EClass && !it.isAbstract()) {
        classes << it
    }
}

ePackage {
    name 'dto'
    nsURI 'urn:example:dto'
    nsPrefix 'dto'

    for (EClass source in classes) {                             ③
        eClass {
            name source.name + 'Dto'
            for (attribute in source.getEAllAttributes()) {
                eAttribute {
                    name attribute.name
                    eType attribute.getEType()                   ④
                }
            }
        }
    }
}
```

① A relative URI is resolved against the script's own URI, so a sibling file is its name. The resource is loaded through the script's resource set, with every resource factory and handler it has, Groovy scripts included.

② Every element of the library resource, collected into a list: the concrete classes.

③ A plain loop over the list. Inside it, `eClass` is type dispatch on the package being built.

④ An element found through the loaded model, the attribute's type, used directly as a reference value.

Listing 12. What the derivation produced

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource dto = resourceSet.getResource(uri("derived.ecore.groovy"), true);
EPackage dtoPackage = (EPackage) dto.getContents().get(0);
assertThat(dtoPackage.getEClassifiers())
    .extracting(c -> c.getName())
    .containsExactly("LibraryDto", "BookDto", "AuthorDto"); ①
EClass bookDto = (EClass) dtoPackage.getEClassifier("BookDto");
assertThat(bookDto.getEStructuralFeatures())
    .extracting(f -> f.getName())
    .containsExactly("title", "year");
assertThat(resourceSet.getResources()).hasSize(2); ②
```

① A class per concrete class of the library, with its attributes.

② Both resources are in one resource set, which is what lets the derived model refer to the elements of the library.

A resource already loaded in the set is returned as it is, not loaded again.

6.2. Iterating a loaded resource

`resource.allContents` and `element.eAllContents()` are EMF tree iterators, and they hide a trap. EMF's tree iterator keeps its stack of iterators in a list, so the object is both an iterator and a collection, and Groovy's collection methods do not know which to use: applied to it, `findAll` and `each` iterate the stack, which starts empty, and find nothing, and `toList()` was seen to return every element in one run and nothing in another. Collect the elements with the iterator's own Java method, `forEachRemaining`, or a `while` loop over `hasNext()` and `next()`, as the listing does, and use Groovy's collection methods on the list. `resource.contents`, a plain list, has no such problem.



Found while writing: the module's former readme showed `resource.allContents.findAll { … }.each { … }`, which finds nothing, for the reason above. Consider a DSL helper that returns all the contents of a resource or an element as a list, so that the natural idiom works (Chapter 10).

6.3. Copying prototypes

`copy('<selector>') { }` copies an element and configures the copy as if it had just been created. The structure comes from wherever it is cheapest to author, a diagram, another model, another

script, and the script adds what only code can say:

Listing 13. A branch, copied from the library (`branch.ecore.groovy`)

```
ePackage {
    name 'branch'
    nsURI 'urn:example:branch'
    nsPrefix 'branch'

    copy('library.ecore.groovy#//Library') {   ①
        name 'Branch'                          ②
        eAttribute { name 'address'; eType 'EString' }
    }
}
```

① The prototype: the class `Library` of the library resource, by URI. A relative path, an element or a closure work too, as for any reference.

② The closure configures the copy: a new name, and one more attribute.

Listing 14. The copy and its prototype

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource resource = resourceSet.getResource(uri("branch.ecore.groovy"), true);
EPackage branch = (EPackage) resource.getContents().get(0);
EClass branchClass = (EClass) branch.getEClassifier("Branch");
assertThat(branchClass.getEStructuralFeatures())
    .extracting(f -> f.getName())
    .containsExactly("books", "authors", "address"); ①

EPackage library = (EPackage) resourceSet.getResources().get(1).getContents().get(0);
EReference books = (EReference) branchClass.getEStructuralFeature("books");
assertThat(books.getEType()).isSameAs(library.getEClassifier("Book")); ②
assertThat(library.getEClassifier("Library")).isNotSameAs(branchClass); ③
```

① The copied features, and the added one.

② References out of the copied subtree still point where the prototype's did: `books` is typed by the library's `Book`.

③ The prototype is unchanged.

A copy is a **snapshot**, not a link: later changes to the prototype do not reach it, and changes to it never reach the prototype, until the script runs again. It is deep, the whole containment subtree comes along, and references within the subtree are rewired to the copies, as `EcoreUtil.copy` does.

Inside a builder, the copy is routed into the containment feature that accepts it; at the top level it becomes a root. When several features accept it, the DSL names them and asks for one: `nodes`

`dsl.copy('<uri>')` places a detached copy in a named feature.

6.4. Containment is exclusive

An element of a loaded resource can be a reference value, as the copied `books` reference to the library's `Book` is. Put into a **containment** feature, it would be moved out of its resource. The builder does not do that silently: it wraps the element in a reference type when the metamodel has one, and otherwise reports that the attachment would move it. Copy the element to contain it.

Chapter 7. Saving

A script resource is read-only by default: `save` throws `UnsupportedOperationException`. The script is the source of truth, and there is no general way to turn an edited model back into the script that built it.

A script that knows what saving should mean says so with `onSave`. It can write itself back, changed or not; push the model to the system its data came from; or both.

Listing 15. A script that writes itself back (`journal.ecore.groovy`)

```
ePackage {
    name 'journal'
    nsURI 'urn:example:journal'
    nsPrefix 'journal'

    eClass { name 'Entry' }
}

onSave { source, outputStream, options ->                    ①
    outputStream.write(source.getBytes('UTF-8'))             ②
    int classes = dsl.roots[0].getEClassifiers().size()
    outputStream.write("// Saved with ${classes} class\n".getBytes('UTF-8'))  ③
}
```

① The callback receives the original source of the script, the output stream, and the save options. The last `onSave` registered wins.

② The source, written back.

③ And a line about the model, computed from it: `dsl.roots` are the roots the script created.

Listing 16. Saving to a stream

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource journal = resourceSet.getResource(uri("journal.ecore.groovy"), true);
String source = Files.readString(new File(journal.getURI().toFileString()).toPath());

ByteArrayOutputStream out = new ByteArrayOutputStream(); ①
journal.save(out, null);
assertThat(out.toString(StandardCharsets.UTF_8))
    .isEqualTo(source + "// Saved with 1 class\n"); ②
```

① A stream of the caller's: the file is not touched.

② The source, and the line the callback added.

7.1. The truncation trap

Saved to its own URI, `resource.save(null)`, the resource opens the file for writing **before** the callback runs, which truncates it. A callback that writes nothing leaves the script empty. A callback that saves somewhere else, to an issue tracker or a database, still writes the source back to keep the file.

7.2. Patterns

- **Write-back with side effects**: push the model to the system it describes, then write the source back unchanged.
- **Self-extending**: append fragments for elements discovered since the last run, so the next load has them.
- **Read-only**: no callback, and `save` fails; the default, and right for most scripts.

Chapter 8. Errors and telemetry

8.1. Errors

Failures to build or to resolve are reported with the script's URI and the line they were written on, even when the failure happens after the script has run, as an unresolved reference does:

Listing 17. A reference to a class that is not there (`broken.ecore.groovy`)

```
ePackage {
    name 'broken'

    eClass {
        name 'Loan'
        eReference {
            name 'book'
            eType "/eClassifiers[name='Book']"
        }
    }
}
```

Listing 18. The failure names the line

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
assertThatThrownBy(() -> resourceSet.getResource(uri("broken.ecore.groovy"), true))
    .hasRootCauseMessage("Cannot resolve segment '@eClassifiers[name='Book']'"
            + " in '/eClassifiers[name='Book']'") ①
    .hasStackTraceContaining("broken.ecore.groovy:8: Cannot resolve"); ②
```

① What went wrong: the path segment that matched nothing.

② Where: a `DslException` in the cause chain, `<uri>:8: …`, the line of `eType`.

The column is not reported: the Groovy runtime does not expose it.

8.2. Telemetry

Loading is traced with the OpenTelemetry instance current for the unit of work, or the one attached to the resource set:

Listing 19. Watching a script load

```
List<SpanEvent.Started> started = new ArrayList<>();
try (PublishingOpenTelemetry otel = new PublishingOpenTelemetry(
```

```
        OpenTelemetry.noop(),
        new PublishingTracerProvider(TracerProvider.noop(), Runnable::run, 1024));
        Scope scope = Telemetry.makeCurrent(otel)) { ①
    otel.subscribe(new Flow.Subscriber<SpanEvent>() {

        @Override
        public void onSubscribe(Flow.Subscription subscription) {
            subscription.request(Long.MAX_VALUE);
        }

        @Override
        public void onNext(SpanEvent event) {
            if (event instanceof SpanEvent.Started s) {
                started.add(s);
            }
        }

        @Override
        public void onError(Throwable throwable) {}

        @Override
        public void onComplete() {}

    });
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
    resourceSet.getResource(uri("tiny.ecore.groovy"), true);
}
List<SpanEvent.Started> load = started.stream()
    .dropWhile(s -> !s.name().equals("Compiling script"))
    .toList();
assertThat(outline(load)).isEqualTo(resource("tiny-spans.txt")); ②
```

① A publishing instance, current while the resource loads, so the spans can be watched without an SDK.

② The spans of the script, from compiling it on, compared with a file.

For this script,

Listing 20. A small script (`tiny.ecore.groovy`)

```
ePackage {
    name 'tiny'
    eClass {
        name 'Novel'
        eSuperTypes "/eClassifiers[name='Book']"
    }
    eClass { name 'Book' }
}
```

the spans nest as the script does:

Listing 21. Its spans (`tiny-spans.txt`)

```
Compiling script
Evaluating script
  Loading EPackage (line 1)
    Setting name (line 2)
    Setting eClassifiers (line 3)
      Loading EClass (line 3)
        Setting name (line 4)
        Setting eSuperTypes (line 5)
    Setting eClassifiers (line 7)
      Loading EClass (line 7)
        Setting name (line 7)
  Resolving references
    Resolving eSuperTypes (line 5)
```

The supertype is set on line 5, recorded there, and resolved after the script, under "Resolving references", with the line it was written on.

Span	Attributes
`Compiling script`	The URI; the line of a syntax error
`Evaluating script`	The URI and the number of roots; the line of a failure
`Loading <Type>`	`nasdanika.dsl.type`, the type as `<nsURI>#<name>`; `nasdanika.dsl.type.superseded` when it superseded less specific types; the line; `nasdanika.dsl.prototype.uri` for a copy
`Setting <feature>`	`nasdanika.dsl.feature`, `nasdanika.dsl.feature.class`, `nasdanika.dsl.feature.kind` (`attribute`, `containment` or `reference`), the line; `nasdanika.dsl.deferred` when resolved after the script
`Resolving references`	`nasdanika.dsl.references`, the number of deferred references
`Resolving <feature>`	The feature's attributes, and the line the reference was written on
`Loading resource`	`nasdanika.dsl.target.uri`, for `loadResource`

`global` emits a log record, `nasdanika.dsl.global`, with the URI, the type and the line. Resolving the capabilities that compile and evaluate the script is traced too, before "Compiling script".

The spans carry locations and types, never values: a model may hold data that a trace must not.

Chapter 9. A DSL for your metamodel

The Ecore DSL is one binding of the generic machinery to one metamodel. This chapter makes another: a DSL for rule sets, with its own qualifier, `orders.rules.groovy`, and a short form for its evaluators. It takes a metamodel, a handler and a capability factory.

9.1. The metamodel

The metamodel is itself written in the Ecore DSL:

Listing 22. A rules metamodel (`rules.ecore.groovy`)

```
ePackage {
    name 'rules'
    nsURI 'urn:example:rules'
    nsPrefix 'rules'

    eClass {
        name 'RuleSet'
        eAttribute { name 'name'; eType 'EString' }
        eReference {
            name 'rules'
            eType "../../eClassifiers[name='Rule']"
            containment true
            upperBound = -1
        }
    }

    eClass {
        name 'Rule'
        eAttribute { name 'name'; eType 'EString' }
        eReference {
            name 'evaluator'
            eType "../../eClassifiers[name='Evaluator']"   ①
            containment true
        }
    }

    def evaluator = eClass {
        name 'Evaluator'
        'abstract'(true)
        eAttribute { name 'documentation'; eType 'EString' }
    }

    eClass {
        name 'SpelEvaluator'                               ②
        eSuperTypes evaluator
        eAttribute { name 'expression'; eType 'EString' }
```

```
    }

    eClass {
        name 'GroovyEvaluator'
        eSuperTypes evaluator
        eAttribute { name 'script'; eType 'EString' }
    }
}
```

① A rule's evaluator is typed by an abstract class.

② Two concrete evaluators: an expression, and a Groovy script.

It is a dynamic metamodel: loaded at run time, with no generated Java code. The DSL works the same way over a generated one.

9.2. A rule set

Listing 23. Rules, in four forms (`orders.rules.groovy`)

```
ruleSet {                                          ①
    name 'orders'

    rule {
        name 'positive-total'
        evaluator 'order.total > 0'                ②
    }

    rule {
        name 'in-window'
        evaluator('order.date > start') {          ③
            documentation 'Orders inside the promotion window'
        }
    }

    rule {
        name 'priced'
        evaluator { expression 'order.items.?[price == null].isEmpty()' } ④
    }

    rule {
        name 'scripted'
        evaluator('GroovyEvaluator') {             ⑤
            script 'order.items.every { it.quantity > 0 }'
        }
    }
}
```

① `ruleSet` is an entry point, because `RuleSet` is a concrete class of the bound metamodel.

② The **short form**: a value for a containment reference, which the binding turns into an element, an expression evaluator.

③ The short form, then configured by a closure.

④ A closure without a value: the binding's default type, configured by the closure.

⑤ An explicitly named type, configured by the closure.

9.3. The handler

A handler supplies the metamodel to the generic machinery, and may adapt the language:

Listing 24. The handler (`RulesResourceContentsHandler.java`)

```
/**
 * The DSL bound to the rules metamodel, with a short form for evaluators.
 */
public class RulesResourceContentsHandler extends DslResourceContentsHandler {

    private final EPackage rules;

    public RulesResourceContentsHandler(
            Resource resource,
            ResourceContentsHandler<CompiledScript> sourceHandler,
            EPackage rules) {
        super(resource, sourceHandler, rules); ①
        this.rules = rules;
    }

    @Override
    protected EObject create(EReference eReference, Object value) { ②
        if (eReference.getEReferenceType() == rules.getEClassifier("Evaluator")
                && (value == null || value instanceof String)) {
            EClass spel = (EClass) rules.getEClassifier("SpelEvaluator"); ③
            EObject evaluator = EcoreUtil.create(spel);
            if (value != null) {
                evaluator.eSet(spel.getEStructuralFeature("expression"), value); ④
            }
            return evaluator;
        }
        return null; ⑤
    }

}
```

① The metamodel: its concrete classes become entry points, and its classifiers are found by name.

② The hook for the short form: an element for a reference, from an authored value, or from no

value.

③ The default type for a reference typed by the abstract `Evaluator`.

④ The value goes into the default type's main feature.

⑤ Declining: everything else keeps its usual meaning.

The order matters, and it is what makes the four forms work together. A value is tried as a **type name** first, which is why `evaluator('GroovyEvaluator')` names a type; then it is offered to `create`; and only if `create` declines is it read as a **selector**, a path or a URI. So a reference whose handler creates elements from strings no longer takes path selectors: it has become a reference to values, which is the point.

Two more hooks can be overridden: `bindings()`, names added to the script's bindings, which override everything else, for example a client for the system a script loads its data from; and `mark(element, feature, line, column)`, called with where each element and feature value was authored, for hyperlinked diagnostics or navigation back to the source.

9.4. The capability factory

A capability factory decides which files the handler serves, and builds it:

Listing 25. The factory (`RulesCapabilityFactory.java`)

```
/**
 * Contents handlers for files with the {@code rules} qualifier: {@code orders.rules.groovy}.
 */
public class RulesCapabilityFactory extends ReflectiveCapabilityFactory {

    private final EPackage metamodel;

    public RulesCapabilityFactory(EPackage metamodel) {
        super(MethodHandles.lookup());
        this.metamodel = metamodel;
    }

    @Capability
    public Flow.Publisher<ResourceContentsHandler<EObject[]>> rules(
            ResourceContentsHandler.Requirement requirement, ①
            Loader loader) {

        int index = requirement.getQualifierIndex();
        if (!EObject[].class.equals(requirement.getContentsType())
                || index <= 0
                || !"rules".equals(requirement.getQualifiers()[index])) { ②
```

```
            return null;
        }

        Resource resource = requirement.getResource();
        ResourceContentsHandler.Requirement script = ResourceContentsHandler.createRequirement(
                resource,
                CompiledScript.class,
                requirement.getQualifiers(),
                index - 1); ③
        CompletableFuture<List<ResourceContentsHandler<CompiledScript>>> sources =
                loader.loadAll(Requirement.of(ResourceContentsHandler.class, script));

        CompletableFuture<List<ResourceContentsHandler<EObject[]>>> handlers = sources
                .thenApply(list -> list
                    .stream()
                    .<ResourceContentsHandler<EObject[]>>map(source ->
                        new RulesResourceContentsHandler(resource, source, metamodel)) ④
                    .toList());
        return Flows.concatMap(Flows.fromFuture(handlers), Flows::fromIterable);
    }

}
```

① The requirement for a contents handler: the resource, the contents type and the qualifiers of its name.

② Only the `rules` qualifier, with a source qualifier to its right; anything else is declined with null.

③ The chaining step: a compiled script, asked of the qualifier to the right, `groovy`, which the module's own capability provides.

④ The handler, around each source handler. The loader does not block: the method returns a publisher of the handlers, built when the source handlers arrive.

9.5. Putting it together

In an application, the factory is a service: `provides CapabilityFactory with RulesCapabilityFactory;` in the module descriptor, with a constructor that takes no arguments and finds the metamodel itself, usually its generated `EPackage`. The test wires it by hand, because its metamodel is loaded at run time:

Listing 26. Loading a rule set (`BindingTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource metamodel = resourceSet.getResource(uri("rules.ecore.groovy"), true); ①
EPackage rules = (EPackage) metamodel.getContents().get(0);
resourceSet.getPackageRegistry().put(rules.getNsURI(), rules);
```

```
CapabilityLoader loader = CapabilityLoader.of(CapabilityFactorySource.concat(
    CapabilityFactorySource.of(new RulesCapabilityFactory(rules)), ②
    CapabilityFactorySource.serviceLoader())); ③
resourceSet
    .getResourceFactoryRegistry()
    .getExtensionToFactoryMap()
    .put("groovy", new ContentsHandlingResourceFactory(loader, resourceSet)); ④

Resource orders = resourceSet.getResource(uri("orders.rules.groovy"), true);
```

① The metamodel, loaded from its script and registered with the resource set.

② The rules factory, in a source in front of

③ the factories of the module path, which compile Groovy.

④ The `groovy` resource factory, over that loader.

Listing 27. Four rules, four evaluators

```
EObject ruleSet = orders.getContents().get(0);
List<EObject> ruleList = (List<EObject>) get(ruleSet, "rules");
List<String> evaluators = ruleList.stream()
    .map(rule -> (EObject) get(rule, "evaluator"))
    .map(evaluator -> evaluator.eClass().getName())
    .toList();
assertThat(evaluators).containsExactly(
    "SpelEvaluator", "SpelEvaluator", "SpelEvaluator", "GroovyEvaluator"); ①

EObject inWindow = (EObject) get(ruleList.get(1), "evaluator");
assertThat(get(inWindow, "expression")).isEqualTo("order.date > start");
assertThat(get(inWindow, "documentation")).isEqualTo("Orders inside the promotion window");
```

① Three expression evaluators, from the short form, the short form with a closure, and the default type, and the Groovy evaluator that was named.

9.6. Resolvers

A list of packages covers most metamodels. When it does not, when types come from a registry or follow naming conventions, or global objects live somewhere else, a handler can be given a `DslContext.Resolver` instead:

Method	Responsibility
`classByName`, `classifierByName`	A class, or any classifier, for a simple, qualified or URI name
`classByInstanceClass`	A class for a Java class

Method	Responsibility
`candidates`	Concrete types that can wrap a target in a reference feature
`names`	The entry points: decapitalized names of unambiguous classes
`global`, `get`	Registering and finding global objects
`selectClass`, `selectClassifier`	The selected class and the classes it superseded, for telemetry
`create`, `mark`, `bindings`	The hooks of the handler above

`EPackageResolver`, which the handler's package constructor uses, is the reference implementation: subclass it rather than start from scratch.

Part III: Looking Ahead

What is planned for the DSL.

Chapter 10. What comes next

PLANNED:

- **A list of all contents.** A keyword that returns all the contents of a resource or an element as a list, so that Groovy's collection methods work on it without the tree-iterator trap of Chapter 6.
- **Resources found by capability.** Scripts that hold global objects are found and loaded through a capability, a requirement with a namespace URI answered by the modules that contribute models of it, so that a script can refer to `people:pavel` without knowing which file defines it.
- **Editor support.** `@DelegatesTo` on the builders, so that editors that support Groovy complete feature and class names inside a closure.
- **Validation.** The model a script built, validated with EMF's validator after the references are resolved, with diagnostics on the lines they concern.
- **Untrusted scripts.** A restricted profile, with Groovy's `SecureASTCustomizer`, for loading scripts from sources that are not fully trusted.
- **Kotlin.** Kotlin's type-safe builders as a second surface, in a module of its own, for checking at compile time; heavier than Groovy, and slower to start, so opt in.
- **Scripts from diagrams.** Draw.io diagrams turned into models by scripts that load them, as the runtime's Draw.io module makes possible, with the structure drawn and the detail written.

Appendix A: Reference

A.1. Types

Type	Purpose
`GroovyResourceFactoryContributor`	Registers the resource factory for the `groovy` extension, over the shared capability loader
`GroovyCapabilityFactory`	The `compiledScript`, `dsl` and `ecore` contents handlers
`CompiledScriptResourceContentsHandler`	Compiles a `.groovy` source to a JSR-223 `CompiledScript`
`DslResourceContentsHandler`	Evaluates a compiled script against a metamodel; hooks `create`, `mark`, `bindings`; `Factory` and `Marker` capabilities for the generic DSL
`EcoreResourceContentsHandler`	The DSL bound to Ecore
`EPackageResolver`, `AbstractResolver`	Name resolution over a list of packages, with supersession; global objects in the resource set
`DslContext`, `ReflectiveBuilder`	The language: entry points, dispatch, keywords, deferred references
`DslException`	A failure with the script URI and line
`DslTelemetry`	The attribute keys and span names of loading

A.2. Precedence

- In a builder: keywords, then features, then classes.
- At the top level: bindings contributed by the handler, then keywords, then entry points.
- For a reference value: an element as it is; then a type name; then the handler's `create`; then a selector, a path, a closure or a URI.
- For a name: the current element's package first, then all packages; a more specific class supersedes the less specific ones.

Appendix B: Building this book

This book is AsciiDoc in `groovy/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl groovy verify                                # the module, its tests and this book
mvn -pl groovy verify -Dzephyr.editor-notes=false    # without editor's notes
mvn -pl groovy verify -Dasciidoctor.skip=true        # without the book
```

The PDF, the single-page HTML and the EPUB are written to `target/book`; the PDF and the HTML are attached to the module's artifact with the classifier `book`, and the test sources with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/groovy/tests/book`, of the scripts in `src/test/resources/book`, and of the module's `module-info.java`. Every script in the book is loaded by a test. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Binding

A DSL tied to a metamodel by a contents handler and a qualifier: `ecore`, or one of your own.

Builder

The delegate of a closure: the element being built, which receives feature calls, class calls and keywords.

Copy

A deep snapshot of a prototype element, configured like a new one.

Deferred reference

A reference given as a path or a closure, resolved after the script has run, with the line it was written on.

Derivation

A model computed by a script from other resources, run again rather than edited again when its source changes.

Entry point

A top-level keyword created from a concrete class of the bound metamodel: `ePackage { }`.

Feature dispatch

A call named after a feature of the current element, which sets it.

Global object

An element registered with the resource set under a URI, reachable from any resource of the set.

Qualifier

A dot-separated part of a file name, read from the right, which selects the contents handler: `groovy`, `ecore`.

Selector

A path, a URI or a closure that names a reference's target.

Short form

A value for a containment reference that the binding turns into an element.

Supersession

A more specific class winning a simple name over the less specific classes it extends.

Type dispatch

A call named after a class, which creates an element of it and routes it into the containment feature that accepts it.